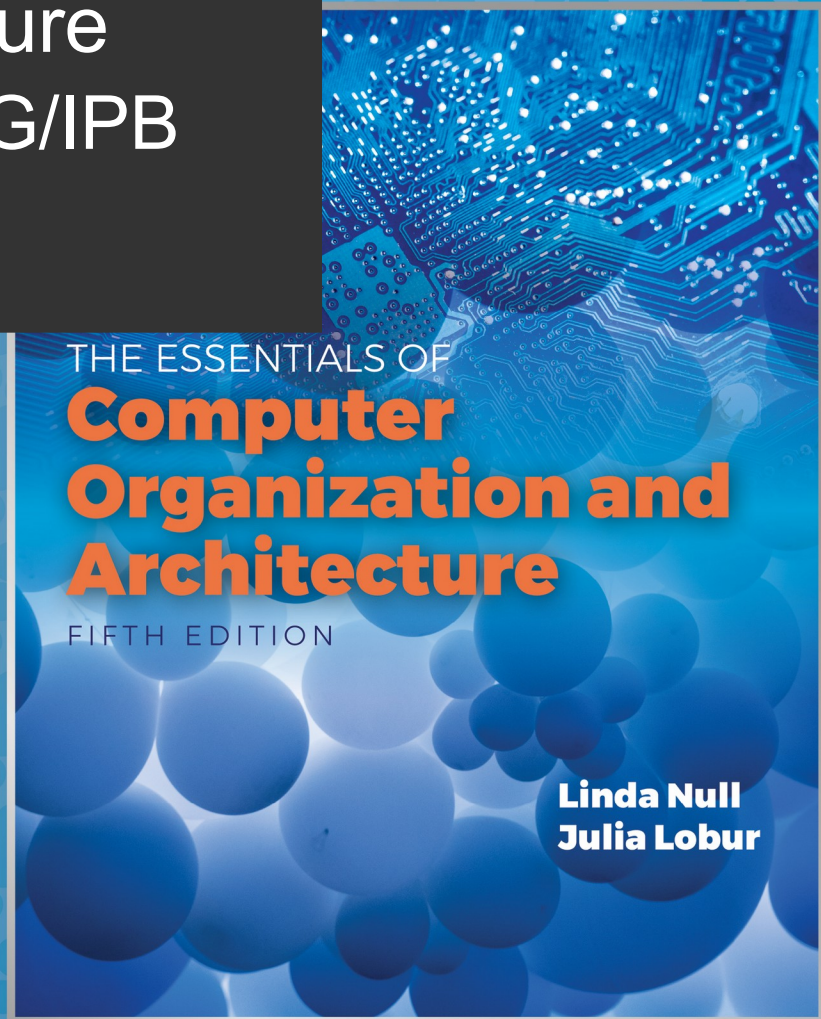


Computer Architecture Informatics Eng., ESTiG/IPB 2025/2026

José Rufino

(mainly based on Chapters 4 and 5 of
the book “The Essentials of Computer
Organization and Architecture”
of Linda Null & Julia Lobur)

Unit 5: Main Memory & CPU Cache



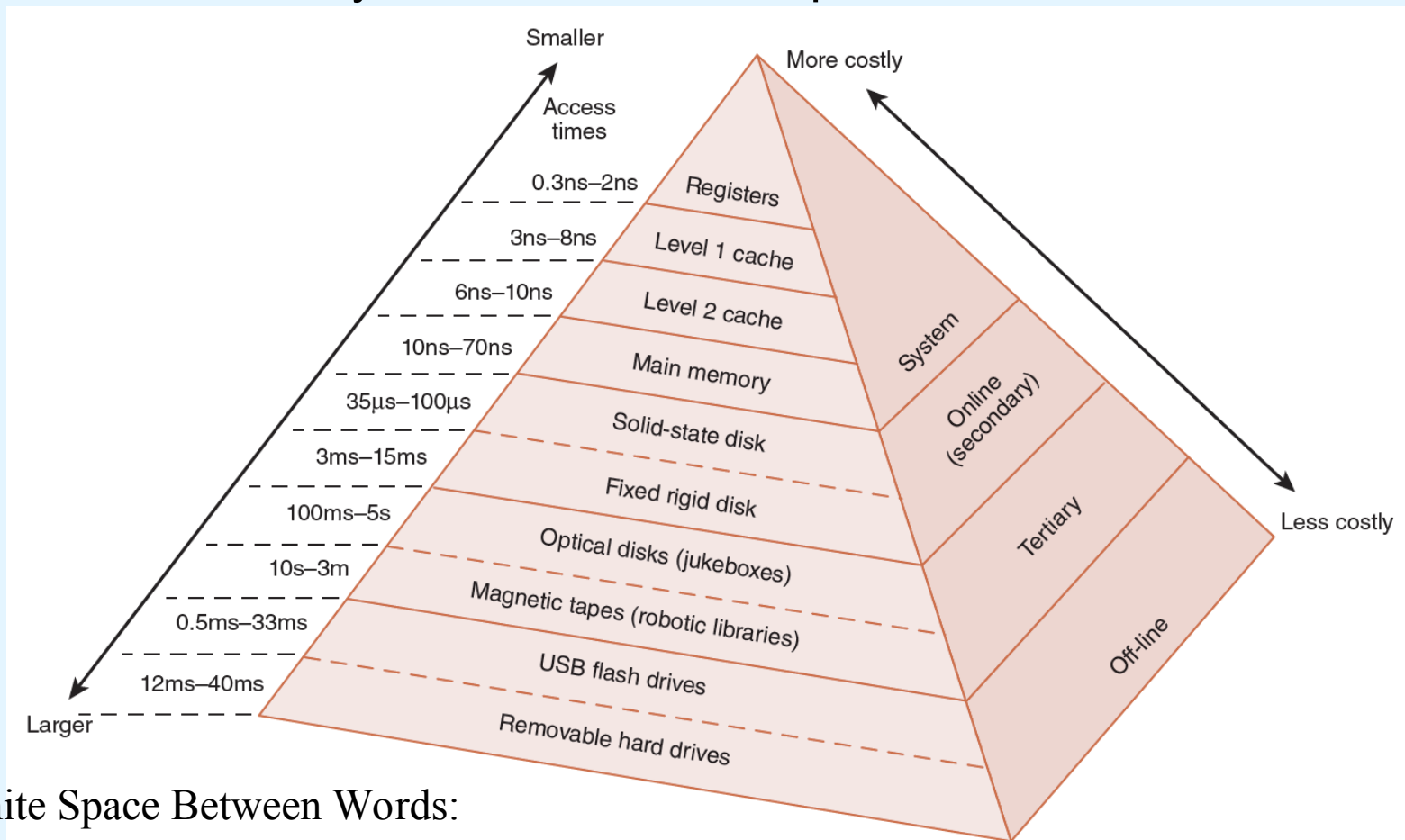
Objectives

- Know the hierarchical storage organization in a computer system, and understand how each level of storage contributes to the system performance.
- Know the types of memories, their building blocks, and its physical and logical organization.
- Master the concepts behind cache memories.
- Know how to assess the impact of cache memories on the performance of programs.

5.1 Storage Hierarchy

- Storage Levels

- Organization according to **speed, capacity, cost, persistence**
 - faster memory is smaller, more expensive and more volatile



The Infinite Space Between Words:

<https://blog.codinghorror.com/the-infinite-space-between-words/>

5.1 Storage Hierarchy

- Storage Levels



- In a computing system, the various storage media are organized hierarchically; this is done in way that maximizes performance, while lowering the cost.
- Small, fast storage is kept in the / close to the CPU.
- Large, slower media accessed through data buses.
- Larger, (almost) permanent storage in the form of disk and tape drives is still further from the CPU.
- Below the registers, accessing an **item** (instruction or data) implies accessing a **block** that contains it.

5.1 Storage Hierarchy

- Hierarchical Access



- To use an item, the CPU needs it in a **register**; if absent, it must be copied there and so the CPU asks the item from the nearest lowest memory level, usually the **CPU cache(s)**.
- If the item is not found in a line (block) of the CPU caches, the **main memory** is queried for that item.
- If the item is not found in a block of the main memory, the request goes to the **secondary storage** layer (SSD/Hard Disk).
- Ultimately, the item may have to be retrieved from **tertiary** near-line optical media, or from **quaternary** off-line tapes.
- Once the item is located, the item and a number of its nearby neighbors in the same block are fetched up in the hierarchy, so that each level ends up behaving as a cache of the lower level

5.1 Storage Hierarchy

- Hit, Hit Rate, Hit Time



- **hit** in a storage level **n**:
 - an intended item is found at level **n**
 - if the item was asked by the upper level **n-1**, it is delivered to it
 - this may involve the item alone (e.g., when a CPU cache delivers an item (data / instruction) of a cache line to the CPU)
 - or may involve the block to which the item belongs (e.g., when the main memory delivers to the CPU cache a memory block)
- **hit rate** in a storage level:
 - percentage (%) of times an item is found at that level
- **hit time** in a storage level:
 - time required to access an item already stored at that level

5.1 Storage Hierarchy

- Miss, Miss Rate, Miss Penalty



- **miss** in a storage level **n**: an intended item is NOT found at level **n**
 - the item must be asked to the lower level **n+1**
 - this may involve the item alone (e.g., when a CPU asks an item (data / instruction) from the CPU cache)
 - or may involve the block to which the item belongs (e.g., when the CPU cache asks the block with that item from main memory)
- **miss rate** in a storage level: **miss rate** = $1 - \text{hit rate}$
 - percentage (%) of times an item is NOT found at that level
- **miss penalty** in a storage level **n**: time required to handle a **miss**
 - includes the time to i) search for the item in the lower level **n+1** and ii) copy the item (or its enclosing block) to level **n**
 - after the missing item/block is copied to level **n**, i) it may be immediately consumed by the original requester, ii) or the original requester repeats the access to level **n** (now with a **hit**)

5.1 Storage Hierarchy

- (Principle of) Locality of References



- ***Temporal locality:***
 - items **recently**-accessed will probably be accessed again in the near future (at least during a limited amount of time, after which the program locality (working set) may change)
- ***Spatial locality:***
 - accesses to a storage medium tend to **focus on certain storage areas** (cache line, memory block, disk block, ...)
- ***Sequential locality:***
 - when items stored **contiguously** are accessed in **sequence** (e.g., access to contiguous cells of a vector, sequential execution of instructions without jumps)

5.1 Storage Hierarchy

- Effective Access Time



- The performance of a hierarchical storage architecture is measured by the **Effective Access Time** (EAT)
- The EAT is a weighted average that takes into account the **hit rate** and the access times to the various levels
- The formula for the EAT must also consider if there are or not simultaneous accesses to several levels
 - **Overlapped** access: an access to a level n happens simultaneously with an access to the lower level ($n+1$); if there is an **hit** in level n , the access to level $n+1$ is aborted; if there's a **miss**, time was spared in the access to level $n+1$
 - **Sequential** access: a level $n+1$ is only accessed after a **miss** happened when accessing level n

5.1 Storage Hierarchy

- Effective Access Time



- **EAT for a two-level storage hierarchy**
 - **Overlapped access:**

$$EAT_2 = H1 \times T1 + (1-H1) \times T2$$

- **Sequential access:**

$$EAT_2 = H1 \times T1 + (1-H1) \times (T1 + T2)$$

- ***H1*** is the **hit rate** of the 1st level of the hierarchy
 - hit rate of the 2nd level (***H2***), not shown, assumed as 100%
- ***T1*** and ***T2*** are the access times to the 1st and 2nd levels

5.1 Storage Hierarchy

- Effective Access Time



- **EAT for a two-level storage hierarchy (example):**
 - with an access time $T_1=10\text{ns}$ to the CPU cache, a hit rate $H_1=99\%$ in the access to that same cache, and an access time $T_2=200\text{ns}$ to the main memory, the EAT is given by:

overlapped access



$$\text{EAT} = \underbrace{0.99(10\text{ns})}_{\text{cache hit}} + \underbrace{0.01(200\text{ns})}_{\text{cache miss}} = 9.9\text{ns} + 2\text{ns} = 11.9\text{ ns}$$

$$\text{EAT} = \underbrace{0.99(10\text{ns})}_{\text{cache hit}} + \underbrace{0.01(\overbrace{10\text{ns}}^{\text{check cache}} + \overbrace{200\text{ns}}^{\text{go to main memory}})}_{\text{cache miss}} = 9.9\text{ns} + 2.1\text{ns} = 12\text{ns}$$

sequential
access



- The EAT expressions for a two-level storage hierarchy can be easily generalized to any number of storage levels (always assuming that the hit rate in the last level is 100%)

5.1 Storage Hierarchy

- Effective Access Time



- **EAT for a 2 vs 3 vs 4-level storage hierarchy:**

overlapped access

$$EAT_2 = H_1 \times T_1 + (1-H_1) \times T_2$$

$$EAT_3 = H_1 \times T_1 + (1-H_1) \times [H_2 \times T_2 + (1-H_2) \times T_3]$$

$$EAT_4 = H_1 \times T_1 + (1-H_1) \times [H_2 \times T_2 + (1-H_2) \times [H_3 \times T_3 + (1-H_3) \times T_4]]$$

sequential access

$$EAT_2 = H_1 \times T_1 + (1-H_1) \times (T_1 + T_2)$$

$$EAT_3 = H_1 \times T_1 + (1-H_1) \times [H_2 \times (T_1 + T_2) + (1-H_2) \times (T_1 + T_2 + T_3)]$$

$$EAT_4 = H_1 \times T_1 + (1-H_1) \times [H_2 \times (T_1 + T_2) + (1-H_2) \times [H_3 \times (T_1 + T_2 + T_3) + (1-H_3) \times (T_1 + T_2 + T_3 + T_4)]]$$

checkpoint: exercises EAT1, EAT2, EAT3

5.2 Types of Memory



- Two main types of memory: **RAM** and **ROM**
- **RAM = random access memory**
 - Also known as *main memory* or *primary memory* (caches are also based on RAM, of a special kind)
 - Supports both **read and write** operations;
volatile content in the absence of electrical charge
 - **Dynamic RAM (DRAM):** used for **main memory**
 - **1 transistor + 1 capacitor (*)** per bit; capacitor leaks charge over time; must be refreshed periodically to prevent data loss
 - many variants: multibank (MDRAM), fast-page mode (FPM), extended data out (EDO), synchronous dynamic RAM (SDRAM), double data rate (DDR) SDRAM, rambus DRAM (RDRAM), direct rambus (DR) DRAM

(*)
DRAM
1-bit cell

5.2 Types of Memory



- **RAM (cont.)**
 - **Static RAM (SRAM):** used for **cache memory**
 - **>= 4 transistors (*)** per bit; keep their state (0/1) if electrically powered (but no need for periodic refresh => faster access)
 - variants: [a]synchronous SRAM, pipeline-burst SRAM
 - **DRAM vs SRAM**
 - DRAM: less components => cheaper, more bit density, thus higher storage capacities; needs more current (for refresh)
 - SRAM: (much) faster, smaller, and more expensive

(*)
SRAM
1-bit cell

RAM Explained - Random Access Memory: <https://www.youtube.com/watch?v=PVad0c2cljo>
(*) How does Computer Memory Work?: <https://www.youtube.com/watch?v=7J7X7aZvMXQ>
Static and Dynamic RAM Explained: <https://www.youtube.com/watch?v=0rNEtAz3wJQ>
SRAM vs DRAM: The Speed Difference between Cache and RAM:
<https://www.youtube.com/watch?v=ammQYLcyebA>

5.2 Types of Memory

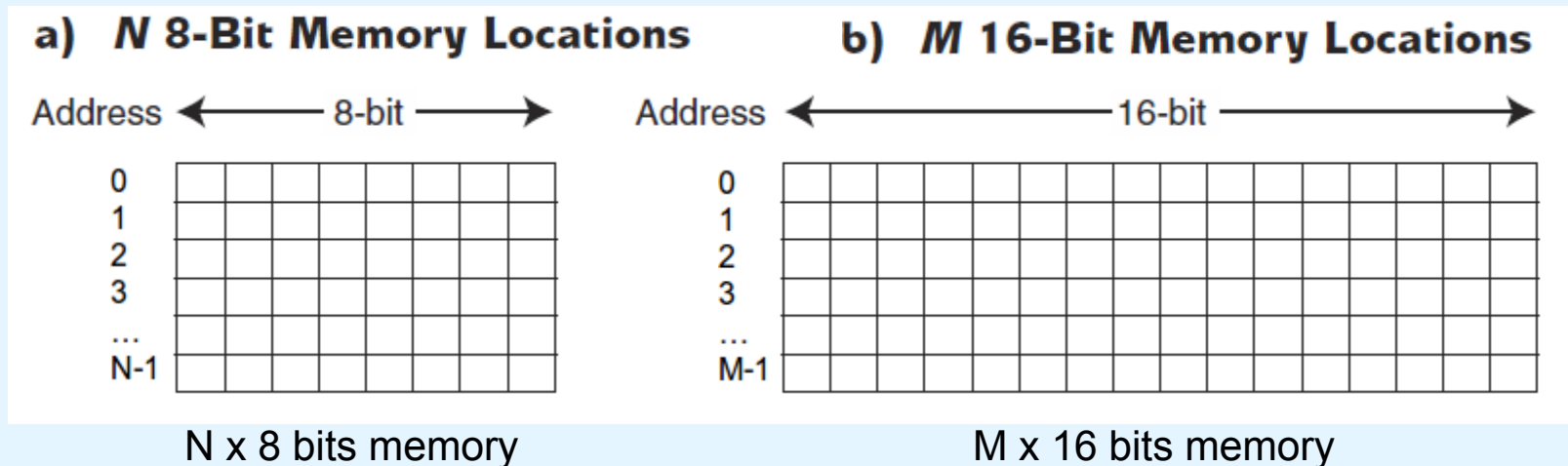


- **ROM = read-only / limited-writing memory**
 - Used to store (semi-)permanent data (e.g., BIOS configuration) that persists when the system turns off
 - Variants
 - **ROM** (Read-Only Memory): allows only reading
 - **PROM** (programmable ROM): with fuses that may be burned once (allowing a single re-programming)
 - **EPROM** (erasable PROM): reprogrammable PROM (with ultra-violet light) several times (but only the full chip)
 - **EEPROM** (electrically erasable PROM): reprogrammable byte-by-byte and in easier way (just with an electric field)
 - **Flash**: reprogrammable EEPROM by blocks (faster)

5.3 Main Memory

- Organization and Addressing

- **Abstract view: Main Memory**=array of (memory) cells
- Each (memory) **cell** = 8 (or a multiple of 8) 1-bit cells
- Each (memory) cell has a **unique address** (integer ≥ 0)
- Abstract view refined: the Main Memory is a **matrix of bits** of order **LxB** (**L** lines of height by **B** bits of width)



5.3 Main Memory

- Organization and Addressing



- One **byte** = 8 bits; One **word** = 1 or more bytes
- **Byte-addressable memory** (storage cell = byte)
 - each byte has its own unique address
 - still, the **instructions** can occupy or manipulate one entire word, and **registers** can also store one word
 - that is: the memory word and the instruction set architecture word may have different dimensions
 - this is the most common scenario in current architectures
 - byte with < address determines the address of the word
- **Word-addressable memory** (storage cell = word)
 - each word has its own unique address
- Addresses and instructions usually fit in one word

5.3 Main Memory

- Organization and Addressing



- When memory is byte-addressable, and the ISA is word-addressable (≥ 2 bytes), **alignment** issues may arise
- Example:
 - reading a 32 bit word in a byte-addressable memory implies that the address of the word is a multiple of 4
- Exception: some architectures allow certain instructions to make non-aligned memory accesses ...
 - i.e., the addresses to access may not be multiple of the word size
 - consequence: may imply reading two memory words to access the equivalent to one (half is in one word, other half in another)

5.3 Main Memory

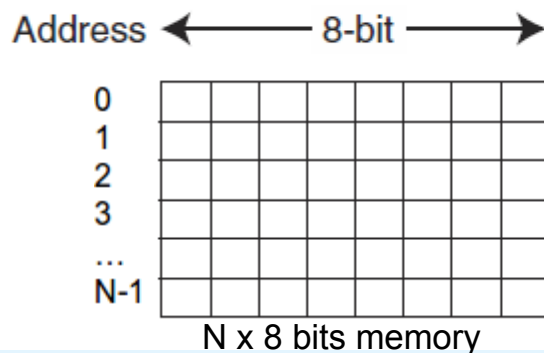
- Organization and Addressing

- In a memory with **C** storage cells
 - cells are numbered (addresses) from **0** to **C-1**
 - the n° of bits of the cell addresses is **$\log_2 C$**

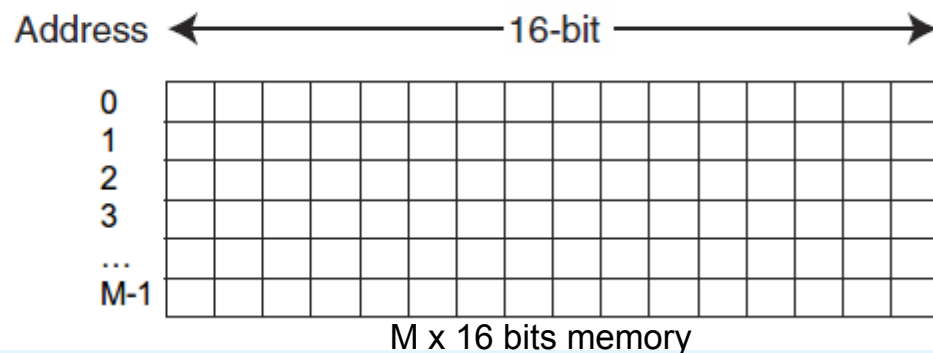
Example:

- **a)** cell=1 byte (byte-addressable); **N** cells; addresses **0** to **N-1** with **$\log_2 N$** bits
- **b1)** cell=1 byte (byte-addressable); **2M** cells; addresses **0** to **2M-1** with **$\log_2 2M$** bits
- **b2)** cell=2 bytes (word-addressable); **M** cells; addresses **0** to **M-1** with **$\log_2 M$** bits

a) N 8-Bit Memory Locations



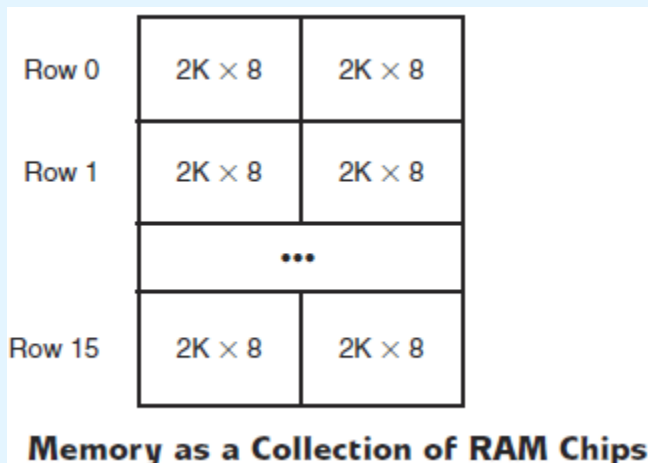
b) M 16-Bit Memory Locations



5.3 Main Memory

- Organization and Addressing

- **Physical view:** a **memory** is based on one or more **modules**, consisting of one or more **chips**
- modules and chips are also (smaller) memories, thus described by the notation **L x B (Lines x Bit Width)**
- **Example 2:** a memory **32K x 16**, with 16 modules **2K x 16**, made with pairs of **2K x 8** chips



Module (RAM stick) with 8 chips
[<https://en.wikipedia.org/wiki/DIMM>]

5.3 Main Memory

- Organization and Addressing



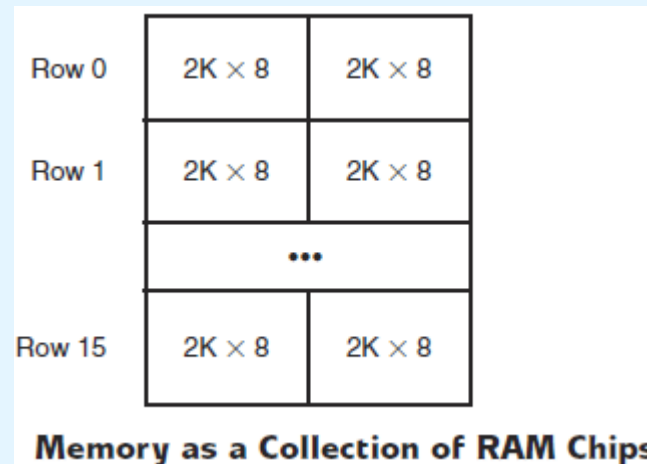
- **abstract (logic) view → physical (real) view mapping:**
 - maps each **memory address** into a certain **module** and a certain **offset** (module **line**) within that module
 - the sequence of bits of an **address** splits into two sets
 - one, specifying the **module** where the address maps
 - another, specifying the **offset** within the module
 - under this subdivision it follows that:
$$\#BITS(\text{address}) = \#BITS(\text{module}) + \#BITS(\text{offset})$$
 - additionally:
$$\begin{aligned}\#BITS(\text{address}) &= \log_2(\#LINES(\text{memory})) \\ \#BITS(\text{module}) &= \log_2(\#MODULES(\text{memory})) \\ \#BITS(\text{offset}) &= \log_2(\#LINES(\text{module}))\end{aligned}$$

5.3 Main Memory

- Organization and Addressing



- **abstract view → physical view mapping:**
 - **Example 2 (cont.)**



- If word addressable (word=16 bits), there are 32K addresses
- Each of those addresses is represented by $\log_2(32K)$ =**15 bits**
- In those 15 bits, 4 determine the module and 11 the offset
 - 16 modules identified with $\log_2(16)$ = **4 bits**
 - 2K offsets (module lines) identified with $\log_2(2K)$ = **11 bits**

5.3 Main Memory

- Organization and Addressing



- **abstract view** → **physical view mapping**:
 - **High-Order Interleaving**: most significant bits specify module, and the least significant bits indicate the offset
 - because memory addresses are mapped sequentially (in order) by the various memory modules, access to a sequence of words (that fits in a module) is done serially
 - **Low-Order Interleaving**: least significant bits specify module, and the most significant bits indicate the offset
 - once memory addresses are mapped rotatively by the various memory modules, access to some words of a sequence may be done in parallel (provided those words being accessed were assigned to different modules)

5.3 Main Memory

- Organization and Addressing

- abstract view → physical view mapping:

– **Example 2 (cont.)** **checkpoint: exercises MP5 to MP9**

#BITS(address) = 15; #BITS(module) = 4; #BITS(offset) = 11; 30722 (10) = 111 1000 0000 0010 (2)
 HOI: 111 1000 0000 0010 => Module = 1111 (2) = 15 (10), Offset = 000 0000 0010 (2) = 2 (10)
 LOI: 111 1000 0000 0010 => Module = 0010 (2) = 2 (10), Offset = 111 1000 0000 (2) = 1920 (10)

Logical Address	Physical Address				
	HOI mapping	LOI mapping	Offset	Module	
0	0	0	0	0	2K x 16
1	1	16	1		
2	2	32	2		
...	...	...	...		
2047	2047	32752	2047		
2048	2048	1	0	1	2K x 16
2049	2049	17	1		
2050	2050	33	2		
...	...	...	...		
4095	4095	32753	2047		
...					...
30720	30720	15	0	15	2K x 16
30721	30721	31	1		
30722	30722	47	2		
...	...	...	...		
32767	32767	32767	2047		

5.4 Cache Memory



- A CPU cache memory speeds up access to RAM items (instructions/data) by storing (copies of) recently used items close to the CPU.
- A CPU cache is much smaller (but much faster) than RAM memory
- Unlike **RAM**, which is accessed by **address**, a **cache** is typically accessed by **content** (thus is called *content addressable memory*)
- The **content** refers to certain **fields** of a memory address
 - these fields are sub-sets of the bits of a RAM address
 - the fields pinpoint the cache line (block) that holds (or should hold) a copy of the memory block to which the RAM address belongs
 - going to the RAM after a memory cell with a certain address, returns to the cache a copy of the block that contains the cell
 - by storing blocks, caches exploit the locality of references

5.4 Cache Memory

- Typical Usage



- **Fields of a RAM address as seen by the cache**
 - **tag** [, **block/set**], **word/offset** ([...] means optional)
- **Usage of the RAM address fields by the cache:**
 - the **block (set)** field tells which cache block (or **set** of cache blocks) holds (**hit**), or should hold (**miss**) a copy of an item
 - the cache block(s) may be valid or not (**miss**), accordingly with the value of a **validity bit** of the block (stored in the block itself)
 - if the block is valid, the **tag** field from the address is compared with the specific **tag** of the block (stored in the block itself)
 - if the tags match (**hit**), the content of the word at a certain **offset** in the cache block, is returned to the CPU
 - if the tags mismatch (**miss**), the missing block is asked to the main memory (where is identified by **tag** [+ **block/set**])²⁶

5.4 Cache Memory

- RAM → Cache Mapping Schemes

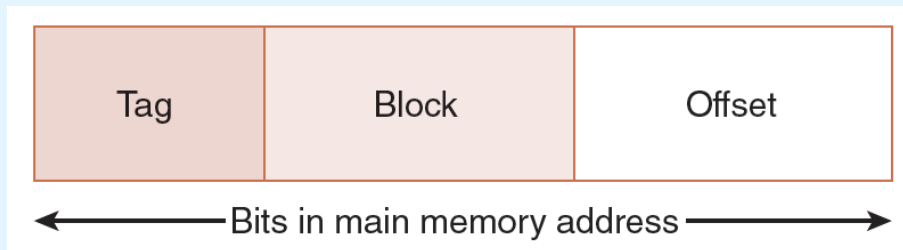


- The division of a memory address into several **fields** provides a **many-to-one mapping** between the larger main memory and the smaller cache memory
 - many blocks of main memory map to the same single cache block; the **tag** field in a cache block allows to know which main memory block was effectively copied there
- **Mapping Schemes:**
 - different ways of dividing a memory address into **fields** + the corresponding cache management algorithms
 - main mapping schemes
 - Direct mapping
 - Fully Associative mapping
 - Set Associative mapping

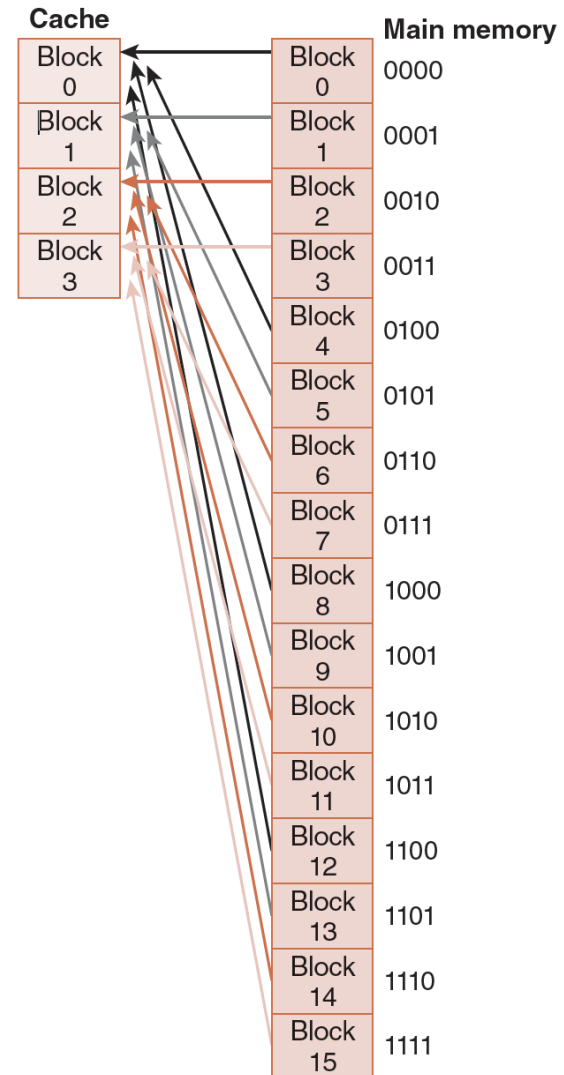
5.4 Cache Memory

- Direct Mapping

- the simplest cache mapping



- in a cache with N blocks, block X of main memory maps to the cache block $Y = X \bmod N$
- example: if $N=4$, block 0 of the cache may hold copies of the blocks 0, 4, 8, 12, ... of memory



16 memory blocks to 4 cache blocks

5.4 Cache Memory

- Direct Mapping



- **cache anatomy** (example):

Block	Tag	Data	Valid
0	00000000	words A, B, C,...	1
1	11110101	words L, M, N,...	1
2	-----		0
3	-----		0

- block 0 contains multiple words from main memory (A, B, C, ...), identified with the tag 00000000
- block 1 contains multiple words from main memory (L, M, N, ...) identified with the tag 11110101
- the other two blocks are not valid (valid bit = 0)

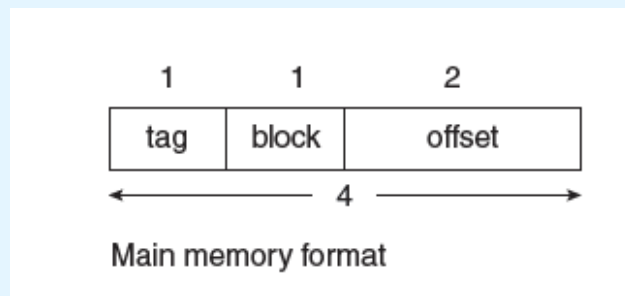
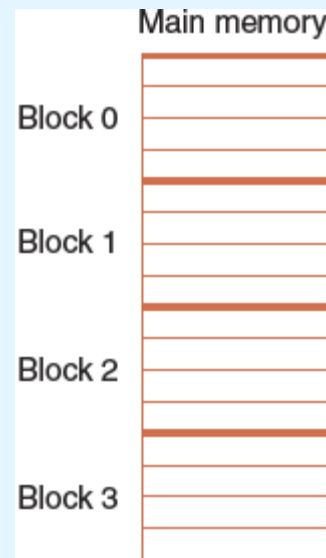
5.4 Cache Memory

- Direct Mapping



- dimensioning of the fields of an address:**

- the size of each field into which a memory address is divided depends on the size of the cache
- **example 1:** i) memory consists of **16** words, ii) cache has **2** blocks, and iii) each block holds **4** words
 - iii) $\Rightarrow \log_2 4 = 2$ bits to address words in blocks
 - ii) $\Rightarrow \log_2 2 = 1$ bit to address blocks in cache
 - i) $\Rightarrow \log_2 16 = 4$, and so $4 - (2 + 1) = 1$ bit left for the *tag*
 - number of memory blocks = $2^{(\#BITS(tag) + \#BITS(block))} = 4$
 = number of memory words / number of block words = $16 / 4$



5.4 Cache Memory

- Direct Mapping



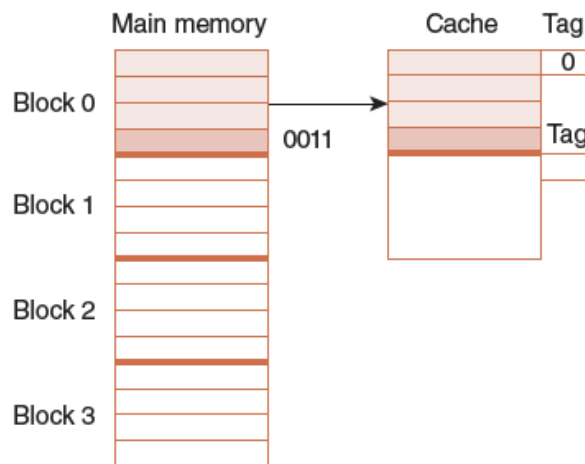
- dimensioning of the fields of an address (cont.):
 - example 1 (cont.):

0	0	1 1
tag	block	offset

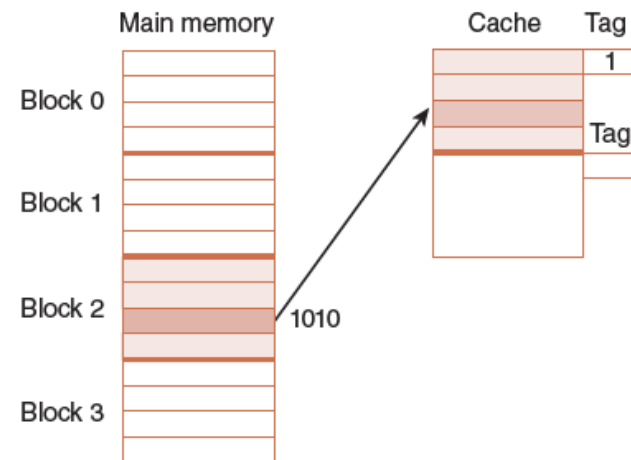
The address 0011 partitioned into fields

1	0	1 0
tag	block	offset

The address 1010 partitioned into fields



Mapping of block containing
Address 0011 = 0x3



Mapping of Block Containing
Address 1010 = 0xA

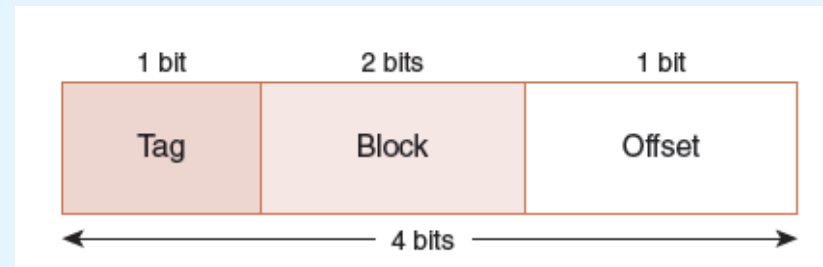
5.4 Cache Memory

- Direct Mapping



- **dimensioning of the fields of an address (cont.):**
 - **example 2:** i) memory consists of **16** words, ii) cache has **4** blocks, and iii) each block holds **2** words
 - iii) $\Rightarrow \log_2 2 = 1$ bits to address words in blocks
 - ii) $\Rightarrow \log_2 4 = 2$ bits to address blocks in cache
 - i) $\Rightarrow \log_2 16 = 4$, and so $4 - (1+2) = 1$ bits left for the *tag*
 - number of memory blocks = $2^{(\#BITS(tag)+\#BITS(block))} = 8$
= number of memory words / number of block words = $16/2$

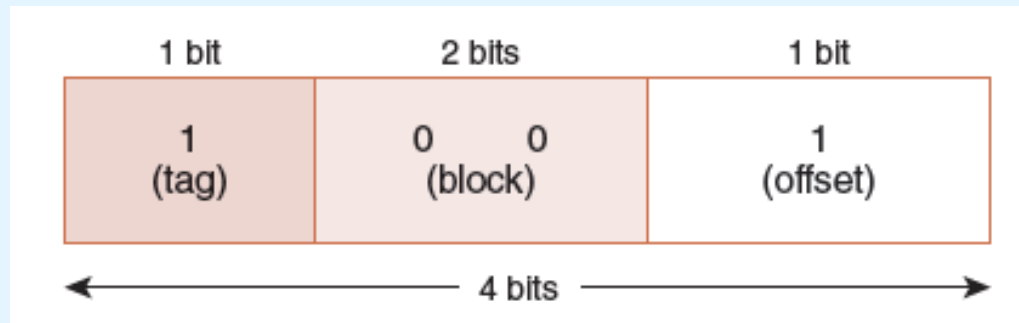
Main Memory	Maps To	Cache
(000) Block 0 (addresses 0x0, 0x1)	→	Block 0 (00)
(001) Block 1 (addresses 0x2, 0x3)	→	Block 1 (01)
(010) Block 2 (addresses 0x4, 0x5)	→	Block 2 (10)
(011) Block 3 (addresses 0x6, 0x7)	→	Block 3 (11)
(100) Block 4 (addresses 0x8, 0x9)	→	Block 0 (00)
(101) Block 5 (addresses 0xA, 0xB)	→	Block 1 (01)
(110) Block 6 (addresses 0xC, 0xD)	→	Block 2 (10)
(111) Block 7 (addresses 0xE, 0xF)	→	Block 3 (11)



5.4 Cache Memory

- Direct Mapping

- **dimensioning of the fields of an address (cont.):**
 - **example 2 (cont.)** : address 9(10) = 1001(2)



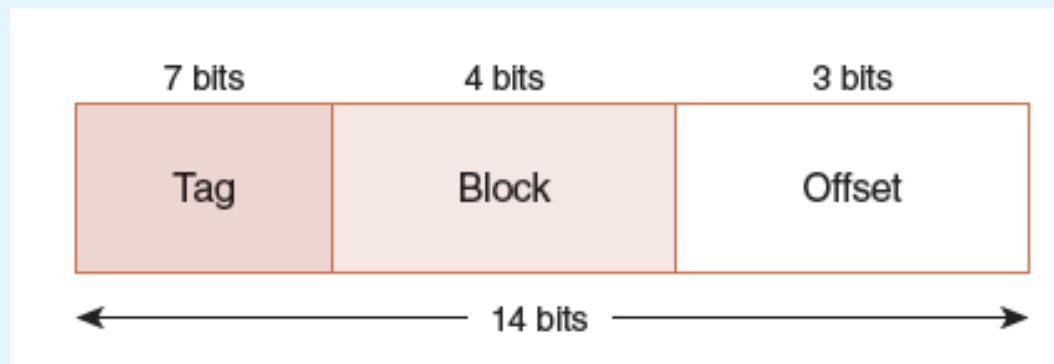
- addresses 0000 (2), 0001 (2), 1000 (2) and 1001 (2) (i.e., 0,1,8 and 9) are always mapped in cache block **00**
- an access to 0(10) also brings 1(10) into the cache
- an access to 8(10) also brings 9(10) into the cache
- what happens for an access stream 0/1, 8/9, 0/1, 8/9, ..., ?

5.4 Cache Memory

- Direct Mapping



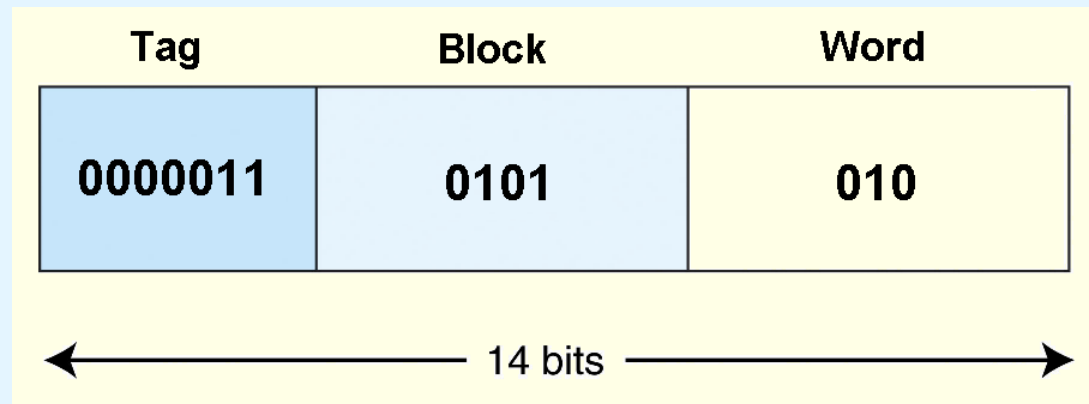
- **dimensioning of the fields of an address (cont.):**
 - **example 3:** i) memory consists of $16K = 2^{14}$ words,
ii) cache has 16 blocks, iii) each block holds 8 words
 - iii) $\Rightarrow \log_2 8 = 3$ bits to address words inside a block
 - ii) $\Rightarrow \log_2 16 = 4$ bits to address blocks inside the cache
 - i) $\Rightarrow \log_2 16K = 14$, so $14 - (3+4) = 7$ bits left for *tag*
 - num. of memory blocks = $2^{(\#BITS(tag)+\#BITS(block))} = 2^{11}$
= num. of memory words / num. of block words = $2^{14}/2^3$



5.4 Cache Memory

- Direct Mapping

- **dimensioning of the fields of an address (cont.):**
 - **example 3 (cont.)** : address 00000110101010 (2) = 1AA (16)



- the address of the 1st word of a block is a **multiple of 8**
- word 1AA belongs to block with words **1A8** a 1AF
=> access to word 1AA copies those 8 words to the *cache*
- any further access to those 8 words will get a cache hit
- what happens for an access stream 1AA, 3AA, ..., ?

5.4 Cache Memory

- Direct Mapping



- **advantages:**
 - direct mapping is the simplest cache approach ...
 - ... and thus cheaper than the other approaches
- **disadvantages:**
 - the mapping of RAM blocks into cache blocks is fixed
 - this inflexibility may prevent the full use of the cache capacity
 - depending on the memory access patterns, the same cache block may be continuously evicted and replaced (thrashing), although there still are unused (free) cache blocks
 - associative mapping (see next) solves this problem

checkpoint: Camera simulator; exercises
CACHE1, CACHE2, CACHE3/EAT4

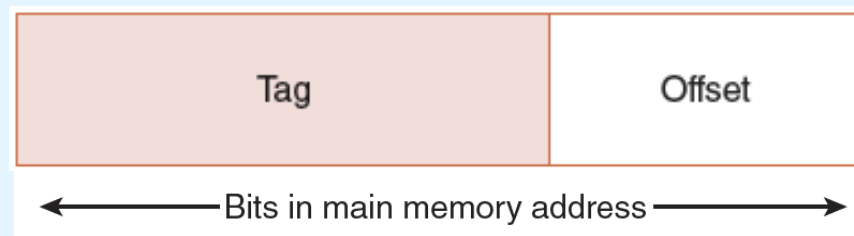
5.4 Cache Memory

- Fully Associative Mapping



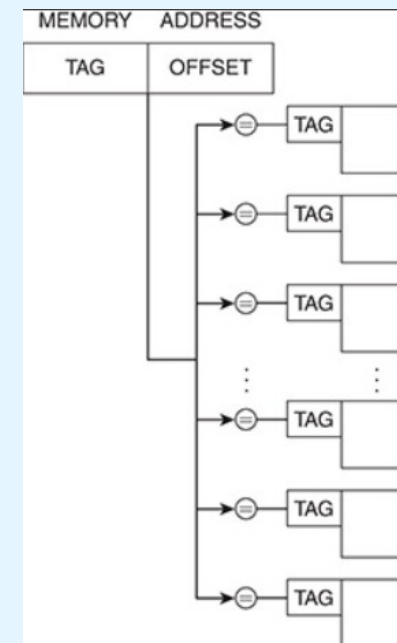
- **principles:**

- a memory block may be copied into any (*) cache block
- a memory address splits only in two fields



- **search in the *cache* for a memory block:**

- now implies searching in all (*) cache blocks
- the *cache* is built using *associative memory*
 - requires special components (more expensive)
 - all *cache tags* compared in parallel with *address tag*



5.4 Cache Memory

- Fully Associative Mapping



- Need for a **replacement policy**:
 - When a fully associative cache is full, to copy a new block to the cache, one must be evicted
 - The block to evict is called the **victim block**
 - There are a number of ways to pick a victim
 - *First-in-First-Out (FIFO)*
 - *Least Recently Used (LRU)*
 - *Least Frequently Used (LFU), etc ...*
 - It is also necessary to consider the need to copy the victim block to the memory if it is “dirty” (modified)

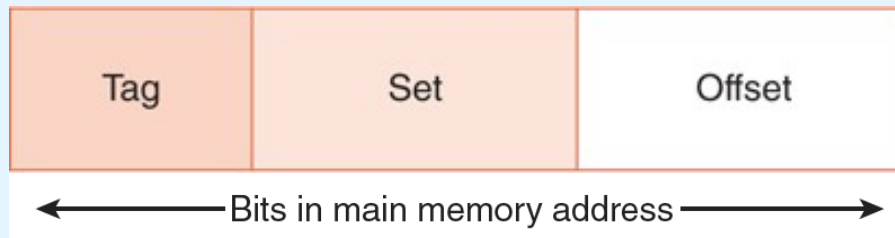
checkpoint: Camera simulator; exercises CACHE4, CHACHE5

5.4 Cache Memory

- Set Associative Mapping

- **principles:**

- combines *direct mapped* and *fully associative* caching
- a memory address is split in three fields:



- the **set** field pinpoints a specific **set of cache blocks**
 - sets have the same number of blocks
- a memory address is always mapped in the same set ...
 - rule adopted from the direct mapping approach
- ... but maps into any of the different blocks of the set
 - rule adopted from the *associative mapping* approach

5.4 Cache Memory

- Set Associative Mapping



- **principles** (cont.):
 - “number of ways, N ” = “number of cache blocks per set”
 - example of a 2-way ($N=2$) set associative cache

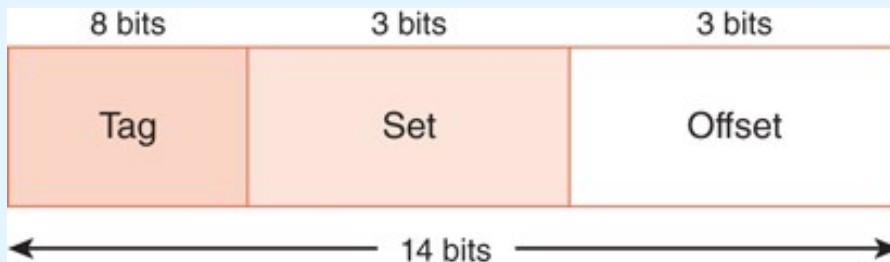
Set	Tag	Block 0 of set	Valid	Tag	Block 1 of set	Valid
0	00000000	Words A, B, C, ...	1	-----		0
1	11110101	Words L, M, N, ...	1	-----		0
2	-----		0	10111011	P, Q, R, ...	1
3	-----		0	11111100	T, U, V, ...	1

- N is an important parameter in cache design
 - a direct mapping cache may be regarded as a set associative cache for the special case where $N=1$
- **tag** and **offset** fields with same role as in *direct mapping*

5.4 Cache Memory

- Set Associative Mapping

- **dimensioning example for $N=2$:** i) RAM consists of 2^{14} words, ii) cache has 16 blocks, iii) each block holds 8 words
 - iii) $\Rightarrow \log_2 8 = 3$ bits to address words inside a block
 - [ii) + $N=2$] $\Rightarrow 16/2=8$ sets in the cache
 - $\Rightarrow$ need $\log_2 8 = 3$ bits for set addressing
 - i) $\Rightarrow 14 - (3+3) = 8$ bits left for the tag field
 - num. of memory blocks = $2^{(\#BITS(tag)+\#BITS(block))} = 2^{11}$
= num. of memory words / num. of block words = $2^{14}/2^3$



checkpoint:

Camera simulator; exercises
CACHE6 to CACHE8/EAT5

How Cache Works Inside a CPU:

<https://www.youtube.com/watch?v=zF4VMombo7U>

5.4 Cache Memory

- Replacement Policies



- *fully associative* and *set associative* caches apply a **replacement policy** when is necessary to evict a cache block
 - *fully associative cache* with K blocks:
 - replacement policy target: any cache block
 - *set associative cache* with N ways:
 - replacement policy target: any of the N blocks of a set
- an **optimal** replacement policy would be able to predict which blocks won't be needed for the longest period of time
- although it is impossible to implement an optimal replacement algorithm, it is instructive to use it as a benchmark for assessing the efficiency of other schemes

5.4 Cache Memory

- Replacement Policies



- the choice of the replacement policy depends upon the locality that we are trying to optimize (usually, we are interested in temporal locality)
- ***least recently used (LRU)***
 - keeps track of the last time a block was accessed and evicts the block unused for the longest period of time
 - disadvantage: > spatial and temporal complexity
 - maintaining an access history for each block takes some storage of the cache and slows down cache

5.4 Cache Memory

- Replacement Policies



- ***first-in, first-out (FIFO)***
 - replaces the block that has been in the cache the longest, regardless of when it was last used
 - popular cache replacement policy; simpler than LRU (same spatial complexity and less temporal complexity)
- ***random replacement***
 - the block to replace is picked at random (simple approach)
 - random replacement can certainly evict a block that will be needed often or needed soon, but it never thrashes (**trashing**: a block X is replaced just to be needed right away, and this pattern repeats, thus thrashing the cache)

Cache Replacement Policies: https://www.youtube.com/watch?v=P_UYI23DddU

5.4 Cache Memory

- Inefficacy of the Cache



- caches are only effective if programs exhibit *locality*
- **example with bad locality in the access to 2D arrays:**
 - programming languages/compiler store 2D arrays in memory differently: C → line-by-line; Fortran → column-by-column
 - if a C program accesses a 2D array by line, an entire line (or several contiguous lines) may fit into the cache; so, after the miss caused by the 1st access, there will/may be many hits
 - if the same program accesses the array by column, it will jump between lines in RAM, incurring in many successive misses
 - example: 5x4 array; cache able to store only one line; access by line => 5 cache-misses; access by column => 20 misses

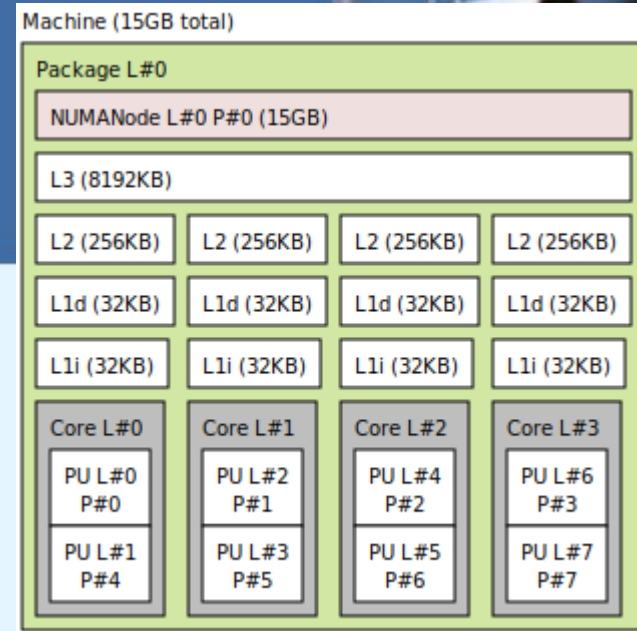
C/C++ compilers: row-major

Fortran compilers: column-major

5.4 Cache Memory

- Inefficacy of the Cache

- **example with bad locality in the access to 2D arrays – real scenario:**
 - output of `lstopo` in a Linux system:
(in Windows, execute the app CPU-Z)
 - to ensure that a 2D array doesn't fit into the biggest cache (L3), it must occupy more than 8192KB = 8MB; a 2D array of 4096 x 4096 cells of 1 byte each will occupy an overall space of 16 MB
 - the access to the array **by line** is 4 times faster than **by column**:



```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    char *a=(char*)malloc(4096*4096);
    for (int i=0; i<4096; i++)
        for (int j=0; j<4096; j++)
            a[i*4096+j]=0; // access by line: for each line, iterate all columns
            //a[j*4096+i]=0; // access by column: for each column, iterate all lines
}
```

```
$ time ./example.exe
real 0m0,058s
```

```
$ time ./example.exe
real 0m0,210s
```

5.4 Cache Memory

- Write Policies



- Cache replacement policies must also take into account **dirty blocks** (blocks that were updated while sitting in the cache).
- Dirty blocks must be written back to memory.
A **write policy** determines how this will be done.
- **write through** policy:
 - updates cache and main memory simultaneously on every write
- **write back** policy (also called **copyback**):
 - updates memory only when the block is selected for replacement

5.4 Cache Memory

- Write Policies



- **(dis)advantages of write through**
 - (-) memory must be updated with each cache write, which slows down the access time on updates
 - this slowdown is usually negligible, because the majority of accesses tend to be reads, not writes
 - (+) cache and main memory are kept coherent
- **(dis)advantages of write back**
 - (-) memory not always agree with the cache (this may cause coherence problems in concurrent systems)
 - (+) memory traffic is minimized

CPU Cache Write Policies: <https://www.youtube.com/watch?v=wfVy85Dqiyc>

5.4 Cache Memory

- Unified vs Separate Caches



- The cache discussed so far is a ***unified*** or ***integrated*** cache where both instructions and data are cached.
- Many modern systems employ **separate caches** for data and instructions. This is called a *Harvard* cache.
- The separation of data from instructions provides better locality, at the cost of greater complexity.
 - Simply making the cache larger provides about the same performance improvement without the complexity.

5.4 Cache Memory

- Multilevel Caches



- Today's systems mostly use **multilevel cache hierarchies**.
 - The cache levels form their own small memory hierarchy.
 - **Level 1** cache (8KB to 64KB) is situated on the processor itself. Today, each CPU-core has its own L1 cache, and it is typically separated. Access time is typically about 4ns.
 - **Level 2** cache (64KB to 2MB) started off-die, but today each CPU-core has its own L2 cache, typically unified. Access time is usually around 15 – 20ns.
 - **Level 3** cache (2MB to 256MB), or upper level caches, are usually off-die, and shared by the various CPU-cores.



5.4 Cache Memory

- Inclusive vs Exclusive Caches

- Another thing to consider is whether an item (data or instruction) can exist in more than one cache level.
- If the cache system is ***inclusive***, the same item may be present at multiple levels of cache.
 - ***Strictly inclusive*** caches guarantee that all items in a smaller cache also exists at the next bigger cache (this was somehow assumed until now).
- ***Exclusive*** caches permit only one copy of an item.
- The trade-offs in choosing one alternative over the others involve weighing the variables of access time, memory size, and circuit complexity.

Cache Hierarchy: How Modern CPU Caches Are Organized (L1, L2 and L3):

https://www.youtube.com/watch?v=7yrK_9PderQ

Conclusions



- Computer storage is organized in a hierarchy, with the smallest, fastest memory at the top and the largest, slowest memory at the bottom.
- There are memories tailored mostly for reading (ROMs), and others that equally support reading+writing (RAM)
- Memories are built using modules that combine several chips of the same capacity; the total capacity of a memory is aggregated capacity of its building blocks
- The addresses of a memory are distributed differently by the modules, with two main approaches currently used (high-order and low-order interleaving)

Conclusions



- Main memory is also organized in equal size blocks, with all blocks having the same number of words
- A cache maps blocks of main memory to blocks of the cache itself, keeping copies of the mapped blocks, with the goal of reducing their access time
- There are three general types of cache: Direct mapped, Fully Associative and Set Associative.
- In the associative caches, block replacement policies are necessary; these may be based on the LRU, FIFO and LFU algorithms; these policies may also need to account for the need to update the memory blocks

References

- book ECOA (6th Edition) – Chapter 4
(Memory Organization and Addressing)
- book ECOA (6th Edition) – Chapter 6
(Memory Hierarchy and Cache Memory)
- <https://www.youtube.com/@BitLemonSoftware/videos>
(in this channel, watch also the video **Cache Miss Types**
https://www.youtube.com/watch?v=YXkfmyb0_eA&t=1s)